Epistack: a multiplayer epistemic commons where agents are first-class contributors

Christopher Bannon
UC Berkeley bannon.c.35@berkeley.edu

July 19, 2026

Abstract

Deep research tools output a narrative summary. A reader cannot verify a summary claim by claim. Two summaries of the same question do not merge.

Epistack replaces the document with a commons. The commons is an append-only claim graph. People and AI agents build the graph together. Every node carries a receipt. This paper reports two contributions.

(1) A deep research pipeline in which code enforces the method, not the prompt. The harness enforces coverage quotas, verbatim quote checks, and a second, adversarial research pass. Every claim carries a receipt. The receipt records the source, the quote, the method, and the person who asked.

(2) A platform that applies the GitHub model to knowledge. Investigations run in real-time multiplayer rooms. External agents connect over MCP and use the same write path as the resident agent. Contributors fork knowledge, challenge claims, merge changes under review, and release citable versions with a JSON export API. The commons records disagreement and never deletes it. The system applies trust at read time, not at write time.

1 Introduction

The contest asks for workflows that convert messy evidence into a form a person can reason with. The contest names three layers: ingestion, structure, and assessment. Current deep research tools cover these layers and then flatten the result into prose. A narrative summary has three structural problems:

1. **A reader cannot verify it.** The reader gets conclusions with citations. No sentence links to the exact passage that supports it. To verify one claim, the reader must redo the research.

2. **A reader cannot build on it.** Two summaries of the same question do not merge, diff, or deduplicate. The work does not accumulate.

3. **It hides gaps.** A well-written summary looks the same after a thorough search and after a shallow search. Open questions appear only if the author includes them.

Epistack is a working system. It runs on Next.js and Postgres and includes an embedded research agent named *eve*. The output of Epistack is not a document. The output is a *commons*. The commons is a shared graph of sources, claims, hypotheses, and open questions.

Every node carries a receipt. The receipt records who wrote the node, what method the writer used, and what evidence the writer used. The paper describes two contributions:

Part 1 (§4). A deep research pipeline that improves on standard deep research tools at all three layers. The gain does not come from a better model. The gain comes from rules in code. The harness enforces these rules. The model cannot bypass them. The rules are: stance-varied search across committed "avenues," full-text read quotas, and quote verification in code. A second research pass attacks the claims from the first pass.

Part 2 (§5). A platform model for where research output should live. Investigations run in multiplayer rooms. The rooms apply the GitHub mechanics to knowledge: fork, merge request, review, and release. AI agents are first-class contributors. They mint credentials, join rooms, and write through the same code path as the resident agent. They appear in the same presence layer as people.

Both parts use the same data model. §2 states the vision that motivates the design. §3 describes the data model. §6 demonstrates the system on the three contest case studies. §7 states the current limitations. §8 compares the system with existing approaches.

2 Vision

Within a few years, agents will produce most research. An agent reads a source, extracts claims, and tests them in seconds. A team of agents runs thousands of investigations in parallel. Human reading speed does not grow. Documents fail at this scale. A person cannot audit a million pages of agent prose. Two million pages do not combine into anything.

Agents will also *consume* most knowledge. When a person asks an assistant about diet or a lab leak, the assistant reasons from the open web. The open web is the worst available understanding of a contested topic. The best understanding

is locked away: in expert heads, in paywalled literature, in long debate transcripts. There is no channel from the best understanding to the assistant.

A model for this channel already exists. Agent skills package expertise into a file that any agent can load. Registries such as skills.sh distribute them. A team writes down its best practice once. Every agent that loads the skill then works at that level. The skill does not document the expertise. The skill installs it. No equivalent exists for knowledge about the world.

Epistack is that equivalent. A topic is a living, versioned artifact: the maintained graph of what is known about one question. The people closest to the question maintain it, the way a package maintainer maintains a library. They investigate in rooms, cut releases, and publish. The topic's MCP server is the install line. Any assistant then answers from the best available understanding, not from the web. Skills democratized craft. Epistack democratizes epistemic state: one person's assistant gets the understanding of a well-staffed research team.

One difference from skills matters. A skill ships know-how, and know-how fails loudly: wrong advice breaks the build. Contested knowledge fails silently. A wrong claim just sits there and looks true. So the published artifact must carry its evidence, its disputes, and its open questions. A topic without receipts is an opinion with an install line. This is why the artifact needs the commons data model, not a markdown file.

Each layer of the system answers one scaling question:

- *Receipts scale verification.* Code checks each quote at write time. A person audits by sampling receipts, not by rereading sources.
- *Content addressing scales contributors.* A thousand writers who find the same fact produce one node with a thousand receipts. More contributors deepen the graph. They do not flood it.
- *Late-binding trust scales openness.* A write-time gate becomes a bottleneck when writers outnumber moderators. Epistack has no write-time gate. Each reader applies their own filter at read time. This is why agents are first-class contributors: the commons accepts work from any number of agents and stays auditable.
- *Releases and MCP scale distribution.* A team investigates once. Millions of assistants consume the result at zero marginal cost. Consumption loops back: a consumer who finds a problem files a challenge on the record itself.
- *The platform scales maintenance.* Formats for structured knowledge have existed and died. They died because they had no venue. Maintainers need fork, merge, review, and release to do the work, and multiplayer rooms to do it together with agents. A format produces artifacts. A platform produces maintainers.

The end state: every question that matters has a maintained topic. Teams of people and agents keep each topic current, the way open-source teams keep software current. Every assistant answers from released, receipted knowledge. GitHub is where teams build software. Skills are how agents load craft. Epistack is where teams build knowledge, and how agents load it.

node / row	layer	keyed by
`sources`	ingestion	hash of stored full text
`claims`	ingestion	hash of normalized text
`mentions`	ingestion	claim × source + quote
`relations`	structure	typed claim→claim edge
`hypotheses`	structure	competing answers
`cruxes`	structure	open questions on a claim
`assessments`	assessment	challenge / credence / endorse
`contributions`	(spine)	one receipt per write

Table 1: The commons schema. Every row in the first seven tables points to a row in the eighth.

3 The commons

The commons is one Postgres schema with a small set of node and edge types. Two invariants apply to every table.

Invariant 1: append-only. The system does not update or delete any row in the commons. To correct a row, a contributor adds a new row with a receipt that `supersedes` the old row. The system records disagreement as a challenge and never removes the challenge. The system derives a state such as "contested" at read time and does not store it. Thus a reader can rebuild any past state of the commons: the reader caps reads at a timestamp. Releases (§5.4) use this property directly.

Invariant 2: late-binding trust. The system accepts every write and attaches a receipt. The system has no write-time gate, no moderation queue, and no reputation threshold. Trust is a read-time operation. Each node records who wrote it and how. Thus a reader can weight contributors as the reader chooses. A lens, which is a saved read-time filter, can apply the same weights.

The COVID-origins debate shows why this matters. The same evidence base gives posterior odds that differ by many orders of magnitude. The result depends on whose judgments the reader trusts. Therefore the commons must store attributed judgments, not one selected answer.

The node types map to the three layers of the contest:

- *Ingestion:* the system keys each row in `sources` by the hash of the stored full text. The stored text is the corpus for quote verification. `claims` are single standalone sentences. The system keys each claim by its normalized text and finds duplicates by embedding similarity. A `mention` links a claim to a source and holds the exact quote. The mention is the receipt that the source states the claim.
- *Structure:* `relations` are typed edges between claims (`supports`, `contradicts`, `depends_on`, `refines`).

`hypotheses` are competing answers. A claim links to a hypothesis with a polarity and a 0–1 *diagnosticity* weight. `cruxes` are open questions on a claim. Each crux states what an answer would change.

- *Assessment:* one `assessments` table holds endorsements, challenges, and credences. Each challenge has a type: counter-evidence, rival interpretation, or methodological objection. Each credence is a 0–1 value, has an attribution, and has a timestamp. The system derives the community credence on a hypothesis as the mean of each assessor's latest value. The full history remains available as a belief timeline.

The `contributions` ledger (Table 1) is the spine under all tables. The ledger holds one row for each write. Each row records the contributor, a versioned method string (for example `record_claim@1` or `claim-pressure-test@1.2`), a SHA-256 hash of the payload, and an optional signature. Each row also records the session and the turn that produced the write. Turn attribution resolves an agent write to the person who asked. A receipt reads: "recorded by eve, during a turn asked by *person*, in *investigation*."

Agent work is never anonymous. The system never attributes agent work to a human who did not do the work. The full chain of custody for a node includes the creation receipt, the quote receipts for each source, the challenge threads, and the derived dispute state. One query returns this chain. The UI renders the chain as a provenance panel on every node.

4 Part 1: the research pipeline

Epistack runs deep research in two forms with one method. The interactive agent follows the method as instructions and skills. The *delegated* runner implements the same loop as a state machine in code. This section describes the delegated form because it makes the stronger claim. Code enforces every step. Thus the method holds even when the model deviates.

A room member submits a brief. An example brief is "find the strongest evidence against X." eve executes the run in the background. eve streams narration into the shared room. At the same time, nodes appear in the graph.

4.1 The loop

```
plan        1 model call: avenues + queries
research    run every query (no model call)
read        1 source/step: fetch FULL text,
            extract quote-backed claims
pressure    1 claim/step: crux questions
probe       1 question/step: search, read,
            edge answers back to the claim;
            unanswered -> open crux
synthesize  1 model call: typed relations +
            avenue-by-avenue accounting
```

Each step makes at most one model call. One HTTP request drives one step. The system persists the cursor after every step. This design has an epistemic function, not only an operational function.

An interrupted run leaves a valid partial record. Every claim that the run wrote has its receipts. The system reports a stalled run as *interrupted*. The system never reports a stalled run as complete.

4.2 Planning

The planner does not answer the question. The planner pins the terms and the scope of the question. Then the planner commits to a set of **avenues**. An avenue is a domain where evidence can come from. An avenue is not a candidate answer.

The planning schema instructs the model to vary stance across queries. The schema also instructs the model to include the strongest case for the less popular answer. This rule targets a specific failure mode. Naive search fanout samples the majority view from many angles. It then reports volume as coverage.

Avenues are commitments. At the end of the run, each avenue is covered, thin, or empty. The summary must account for all avenues (§4.5).

The interactive skill applies the same method at a larger scale. It uses 4–8 avenues for contested questions. It uses 8–12 base queries that vary in stance and discipline. It adds a persona fanout: the queries that a skeptical specialist, an insider, or an advocate of the less popular answer would run.

Before any web search, the planner queries the existing commons. Nodes that other investigations established are first-class search targets (§5.5).

4.3 Ingestion

The research phase runs the queries of every avenue. General web queries go through a search API. Scholarly queries go through OpenAlex [4], which supplies reconstructed abstracts and peer-review flags. The phase deduplicates URLs across avenues. Then it queues candidate sources.

The read phase then processes one source per step. For each source, the read phase does three actions. (1) It fetches the full text with a readability-grade extractor and a plain-HTTP fallback. It records the retrieval route in the receipt of the source. (2) It stores up to 60 kB of text as the quote-verification corpus of the source. (3) It asks the model to extract at most six atomic claims. Each claim is a standalone sentence with a verbatim quote.

The harness then checks each quote. It folds the quote and the stored source text. The fold normalizes typographic quotes, dashes, and whitespace. The fold does not change words. The harness requires the quote to appear as a substring of the source text. If the check fails, the harness rejects the claim: no verified quote, no receipt, no claim.

This rule was first a prompt instruction. The rule moved into code after observed failures. In those failures, the model paraphrased a source and labeled the result a quote. This move from prompt-enforced rules to code-enforced rules is the design principle of the pipeline (§4.6).

Extraction also records failure. An irrelevant source does not consume the read quota of the avenue. When a fetch fails, eve narrates the failure and skips the source. The system does not hide failed fetches.

The system deduplicates recorded claims by meaning. An embedding-similarity check compares each new claim to the existing graph. The check either creates a new canonical claim, or it attaches a new mention to an existing claim. The new mention carries the quote from the current source. Ten sources that repeat one statement become one claim with ten receipts. This structure lets a reader ask a question that a narrative summary cannot state: how much of this support is correlated?

4.4 Pressure test and probe

Most deep research tools stop after ingestion and write a summary. This pipeline then tests its own output. The **pressure** phase applies a structured pressure test to each recorded claim, up to twelve claims per run. The phase lays the claim out as a causal chain. Then it examines each joint of the chain:

- *baseline:* would the outcome occur anyway?
- *driver:* does the claimed cause connect to the outcome?
- *mechanism:* is the stated reason the real reason?
- *stakes:* is the effect as large as claimed?
- *reversal:* could the effect run the opposite way?

The output of the pressure test is not a verdict. The output is up to two *researchable questions* per claim. Each question carries its implication: an answer would weaken, strengthen, or reverse the claim. Each question also carries a concrete search query. The method restates the argument structure of competitive debate without jargon. The full skill ships with the system and includes a worked example.

The **probe** phase researches those questions. It handles up to sixteen questions per run, one question per step. Each step searches, fetches a source, and extracts up to three answering claims. The verbatim-quote rule applies again, in code.

The system records each answering claim with a typed relation back to the claim under test. The crux question is the rationale for the relation. The system handles the two outcomes differently on purpose:

- If the probe finds evidence, the graph gains claims that support or contradict the parent claim. The system attaches these claims at the correct joint.
- If no readable source answers the question, the system records the question as an **open crux** with the status `searched_unfound`. A search that finds nothing is a finding. Missing evidence becomes a first-class output. The reader sees which questions would move the answer. The reader also sees that the system searched for those questions.

avenues per run	≤ 4
queries per avenue	1–2
candidate sources per avenue	≤ 10
full-text reads per avenue	5
claims extracted per source	≤ 6
claims pressure-tested	≤ 12
crux questions per claim	≤ 2
probe searches per run	≤ 16
stored text per source	60 kB

Table 2: Code-enforced quotas for one delegated run. Interactive runs use larger values. Each number is a parameter. Larger constants make the same machine run deeper with no change in method.

4.5 Synthesis

When the final phase starts, all evidence is already in the graph. The synthesis call may only add structure. It may add up to four typed relations between existing claims. The harness validates each relation against real claim ids. The harness drops an invalid id and does not write it. The call may also add at most one new competing hypothesis.

The harness constrains the summary to an *accounting*. For each avenue, the accounting states the number of sources read and the number of claims recorded. The accounting states which avenues were thin or empty. The accounting states what remains open. The harness supplies the true counts for each avenue. Thus the summary cannot report coverage that the run did not perform.

An empty avenue is a machine-readable target. The next run, or the next person, starts there and does not repeat covered ground.

4.6 Code-enforced rules

The quotas in Table 2 are constants in the runner, not prompt text. The model cannot stop reading early. The model cannot skip the pressure phase. The model cannot record a claim without an existing source. The model cannot record a quote that the source does not contain. Each rule exists because the model violated the prompt-only version in practice.

The method is versioned data. Every receipt names the method version that produced its write, for example `record_claim@1` or `claim-pressure-test@1.2`. When the pipeline improves, old claims remain auditable under the method that made them. A reader can filter claims by method version. An objection can target the outputs of a method, not only the outputs of a person. A knowledge base survives changes to its own tools only when the data records the method.

The division of labor is simple. The model proposes writes. The harness accepts or rejects them. This division makes the pipeline scalable in the sense of the contest.

The human design work is the fixed method: avenue planning, quote verification, and the pressure-test lenses. People do this work once, and it is not a per-case cost. Everything

that improves with better models is a model call behind a schema. Examples are query quality, extraction, and question generation. Everything that improves with more compute is a constant in Table 2.

Runs are concurrent by construction. Multiple delegated runs can work on separate sub-questions of one investigation in parallel. All runs write into one deduplicated graph.

4.7 Comparison with the baseline

The contest instructs judges to compare with off-the-shelf deep research. A frontier deep-research product often writes more polished prose. On the other axes, the structured run is stronger:

- *Verifiability:* every claim links to a verbatim quote. Code checks the quote against the stored source text. A person can verify one claim in seconds.
- *Robustness of the record:* an invented quote cannot enter the graph. The write path rejects it.
- *Missing evidence:* open cruxes with the status `searched_unfound` are explicit outputs. Each open crux states what an answer would change.
- *Coverage:* the avenue accounting separates "searched and found little" from "did not search."
- *Compounding:* the output is a graph. The graph deduplicates into prior work, and the next run can query it. Two narrative reports have no comparable operation.
- *Assessment:* the system attributes and timestamps credences per person. Credences are not embedded in prose. One graph serves readers who weight the evidence differently.

▶ **Demo video –:–** A delegated run: the brief, the narration from eve, nodes that appear in the shared graph, and a recorded open crux.

5 Part 2: the platform

The second contribution states where research output should live. We make this claim: people should build knowledge on contested questions the way teams build software. Software work has teams, lineage, review gates, versioned releases, and machine consumers. The contributors should include AI agents on equal terms.

Epistack implements this claim end to end. The conversational research surface and the collaboration platform are one product. One set of graph primitives serves human readers, human writers, the resident agent, and external agents.

5.1 Rooms

An investigation is a real-time room. The room shows a shared chat with eve on one side. The room shows the live claim graph on the other side. The multiplayer layer has these parts:

- *One durable log.* Every member renders the room from the same replayable server stream of eve's session. The sender renders from the same stream. There is no CRDT and no custom sync protocol. N browsers converge because there is exactly one log.
- *Presence and cursors.* The system broadcasts live cursors, with cursor chat, in graph coordinates. The graph layout is deterministic: a seeded force simulation over sorted inputs. Therefore every client resolves the same coordinates to the same node. A claim that one member points at is the same claim on every screen.
- *Tours.* A member asks `@eve` a question in the graph. The system answers in place, or the system starts a tour. In a tour, an eve cursor moves from node to node, and follower cameras track it. The agent uses the same presence primitives as people.
- *Anchored discussion.* Comment threads anchor to exact quote spans in the transcript. A member can promote a substantive comment into a formal commons challenge. Discussion has a defined path into the receipted record.
- *Delegation.* Any member can submit a background brief (§4). All members watch the run write into the shared graph. The system attributes the run to the delegator.

▶ **Demo video –:–** Two people and eve in one room: cursors, cursor chat, a tour, and a comment that becomes a challenge.

5.2 Agents as first-class contributors

Common practice attaches agents as chatbots or ingestion scripts. Epistack registers an external agent as a *contributor*. A contributor is an identity that writes with receipts, appears in presence, and participates in review. The mechanism is an authenticated MCP server [3]. A signed-in person mints a named agent key. This action creates a contributor row of kind `agent` and a bearer token.

The system shows the token once. The connection is one config block in any MCP client:

```
{ "mcpServers": { "epistack": {
    "url": "https://<host>/api/mcp/agent/mcp",
    "headers": {
      "Authorization": "Bearer esk_..." } } } }
```

The token is the capability. Revocation is a timestamp. The system attributes every write to the agent's own contributor id. The system records the person who minted the key as `on_behalf_of`. Accountability survives delegation. The system does not credit the agent's work to the human.

A connected agent receives three tool bundles (full reference in Appendix A):

```
# read the commons
search, fetch, get_claim, get_hypothesis

# investigations
list_investigations, create_investigation,
get_investigation_graph

# write the graph (eve's own write path)
record_source, record_claim, record_relation,
record_hypothesis, link_claim_to_hypothesis,
record_crux, record_credence, file_challenge

# collaborate as a room member
get_transcript, add_comment, reply_to_comment,
send_message, delegate_investigation,
delegate_step
```

Three design decisions support the "first-class" claim.

One write path for all writers. The MCP write tools call the same code that eve uses, with a contributor parameter. The tools apply the same embedding dedup, the same verbatim-quote enforcement, and the same receipts. An external agent cannot write a claim that eve could not write. There is no second write path with lower integrity.

Agents also get the full method of §4. An external agent can start a delegated run (`delegate_investigation`) and drive it step by step. The agent can also take a conversational turn in a room under its own name (`send_message`).

Agents are visible. Every agent write fires an activity broadcast. Clients derive agent presence from recency: an agent that wrote seconds ago is present. The agent's avatar appears in the same stack as the human avatars, with the label "agent, on behalf of *minter*". A live cursor moves to the node that the agent last touched. There is no separate bot feed. A member watches an agent work in the same way that the member watches a colleague work.

Agents are answerable. Agent writes are ordinary receipted contributions. Members can challenge them, supersede them, and filter them like the contributions of anyone else. A reader who distrusts a specific agent can discount only that agent's contributions. Late-binding trust makes the question "should agents write?" a per-reader setting, not a platform policy.

▶ **Demo video –:–** An agent connected over MCP: the key mint, the agent's avatar and cursor, claims that land with receipts, and a challenge that a person files on one claim.

5.3 Fork and merge

Contested questions need divergence. A group that rejects a framing must be able to take the evidence in a different direction. The group must not need permission for this. The two lines of work must later be able to converge again without copies. Epistack adapts the git model to an append-only graph:

- *Fork.* A contributor can fork any investigation at a point in time. A fork's read scope is its ancestor chain. The system caps each hop at its fork time. A fork of a fork cannot see data from after its grandparent's fork time. Writes always land in the one shared commons. The fork is a view, not a copy.
- *Merge request.* The path back upstream is an analog of a pull request. A fork proposes adoption to an ancestor. The system computes the diff live, and the diff is pure addition. An append-only graph has no "modified" state, so the additions are the diff. The target's graph shows the incoming nodes with a highlight.
- *Review gate.* Only the owner of the target can accept. This gate is the one gate in the system, and it gates scope, not truth. Acceptance copies nothing. Acceptance widens the read scope of the target lineage to adopt the fork's hops. The system freezes the hops at decision time, so later fork-side work does not enter retroactively. The acceptance is itself a receipted contribution.

A merge is scope adoption over one shared graph. The graph is deduplicated and content-addressed. The same claim can belong to many lineages at the same time. Coverage compounds and does not split into rival copies. Wikis and document workflows lack this property.

▶ **Demo video –:–** Fork an investigation, diverge, open a merge request, review the preview of incoming nodes, and accept.

5.4 Releases

A live graph changes over time. A citation needs a fixed target. Any contributor can create a **release**. A release is a named, versioned checkpoint. The release stores a recipe: the scope hops plus a cutoff timestamp. The release never stores a copy of the content.

Immutability follows from the append-only invariant. The same hops at the same cutoff always resolve to the same graph. The resolved graph includes the dispute states and the credences as they stood at that moment. Each release has a public page with no authentication. The page shows a citation card (plain and BibTeX) and a machine-readable export:

```
GET /api/releases/<id>/export
-> { release: { title, version, cutoff,
                citation }, graph: { claims,
     sources, relations, hypotheses, cruxes,
     challenges, credences } }
```

Forks, merge previews, and releases all resolve through one scope algebra. The scope algebra is a list of {sessions, cutoff} hops. There is a single definition of "what this lineage can see, as of when." A release stores only that recipe. The release page therefore continues to render after someone deletes the room that produced the release. A citation remains usable after its room is gone.

This subsystem is the distribution half of the platform. A team investigates in private rooms. The team then publishes versioned artifacts. A human reader, a downstream application, or another agent reads each artifact at its URL. Read-only MCP servers expose the commons, and curated per-topic slices, to any MCP client with no key.

5.5 Compounding

Three mechanisms make separate investigations accumulate into one shared record:

- *Content addressing and dedup* (§4.3): when two rooms record the same claim, the graph stores one node with two receipts.
- *Commons search and seeding:* full-text search spans all investigations. When a member asks a new question, the system injects a digest of prior findings into eve's context. The digest covers what other investigations established. The research planner treats existing nodes as search targets. New work starts where earlier work ended, not from zero.
- *People as an assessment layer:* every avatar opens a person card with that contributor's contribution history. A member can follow contributors and compare beliefs. The system ranks the hypotheses where the latest credences of two people diverge most. These hypotheses are the most productive places for the two people to talk. Assessment stays attached to people, and this attachment makes late-binding trust work in practice.

6 Demonstration

We ran the full workflow on all three contest case studies as separate investigations in one shared commons. The case studies are COVID-19 origins, LHC black holes, and the nutritional value of eggs. The three cases stress different parts of the system. This is intentional. One methodology covers three questions with different shapes, and we added no code for any single case.

- *COVID origins* stresses assessment. The graph carries competing hypotheses. Claims link to each hypothesis with a polarity and a diagnosticity value. Challenges sit on the contested claims that the hypotheses depend on, and contributors record attributed credences. The Rootclaim debate [2] showed this structure, and we built the data model to hold it.
- *Black holes* stresses structure. A settled conclusion rests on a dependency chain (`depends_on` relations). Each link in the chain stays open to inspection. The reader can see what would have to fail before the conclusion changes.
- *Eggs* stresses scouting. The question is open-ended. The main work is to find the important sub-questions. Avenues, pressure tests, and open cruxes carry that work.

We include the investigations, their releases, and the full receipt trails in the submission as a database snapshot. The judge runbook (`README.md`, Appendix C) restores the snapshot locally in a few commands. A judge can open the rooms and inspect the provenance panel for any claim. A judge can run a delegated brief on a new question and can connect an MCP agent to the same graph. A judge can inspect every claim in this paper in the artifact.

▶ **Demo video –:–** The three case-study graphs: hypotheses and credences for COVID origins, the dependency chain for black holes, and the crux map for eggs.

7 Limitations

- *Quote verification proves faithful extraction, not truth.* A verbatim quote from an unreliable source is still unreliable. Challenges, source guarantees (for example, peer-review flags), and attributed credences address reliability. The system verifies the chain of custody. The system does not claim that the chain settles the question.
- *Ingestion has gaps.* Paywalled sources and PDF files fall back to a metadata-only path. The system cannot quote-verify claims from that path against stored full text. The receipt records the weaker guarantee. The coverage gap is real for questions that depend on academic sources.
- *Deduplication uses a threshold.* The system decides claim identity with embedding similarity at a fixed cutoff. The cutoff sometimes merges claims that should stay distinct. The append-only rule limits the damage. A wrong merge adds a mention and destroys nothing, and `refines` edges restore lost distinctions. The threshold remains a coarse test.

8 Related work

Deep research products (OpenAI, Anthropic, Perplexity, and others) share the fanout-read-synthesize skeleton. They do not share the output data model. In those products, claims are not addressable, quotes are not machine-verified, missing evidence is not an output, and two runs do not compound. *Argument mappers*, such as Kialo, have the graph, but people curate the graph by hand. Their nodes are opinions, not quote-receipted extractions from sources. *Wikipedia* compounds and cites sources, but it resolves disagreement into a single text. Its unit is the article, and a reader cannot diff, merge, or credence-weight an article. *Community Notes* [5] demonstrated attributed crowd assessment at scale. It applied that assessment to single items, not to claim graphs. *Structured debate* (Rootclaim [2]) produces rich, contested material with many attributed judgments, and we built our data model to hold that material. The 2024 COVID-origins debate is a dataset that needs this data model. We propose a combination of git mechanics, multiplayer rooms, a receipted claim graph, and agents as accountable contributors. To our knowledge, no other working system combines these parts.

9 Map to the judging dimensions

- *Epistemic uplift:* The system verifies each claim against a verbatim quote. Each node shows the evidence that supports it. The harness shows cruxes and thin avenues explicitly (§4.3–4.5). The commons records attributed credences instead of one selected answer (§3).
- *Generalizability:* One methodology serves all cases. The system needs no engineering for each case. The methodology runs on three cases with different shapes (§6). Avenue planning is the one step that adapts to the shape of a case. This step is a model call, not a hand-built ontology.
- *Compounding and shareability:* The system deduplicates content with content addresses. The system provides commons search and seeding, fork, merge, release, JSON export, and a read-only MCP server (§5.5, §5.3–5.4). The outputs are graphs with receipts, not narrative summaries.
- *Scalability:* Every judgment call is a model call behind a schema. The budgets are constants (Table 2). Runs execute in parallel and write into one deduplicated graph. The process contains no hand-designed human step (§4.6).
- *Methodological transparency:* The methodology ships as readable skills and a state machine. Every write names its method@version. The methodology is therefore auditable, versioned data (§4.6).
- *Adversarial robustness:* The write path checks each quote in code and rejects invented quotes. The harness runs adversarial tests on every claim as a required step. Each challenge has a type and a receipt. No contributor can delete a challenge. §7 states the failure modes.
- *Insight contribution:* The insight is the reframe itself. Research output becomes a multiplayer commons with git mechanics. Agents become accountable contributors, not tools (§5).

10 Conclusion

Epistack changes the unit of research output. The old unit is a document that one intelligence produces. The new unit is a commons that human and artificial intelligences maintain together.

Part 1 shows how the system makes the write path trustworthy. The harness enforces coverage. The code checks each quote. The harness runs adversarial tests on each claim. The system attaches a receipt to every write.

Part 2 shows how the system makes the commons social and cumulative. Contributors fork, review, merge, and release. Humans and agents work in one room, with one identity model and one write path.

The system scales along stated parameters. More compute makes runs wider and deeper. Better models improve every judgment call behind a schema. More contributors, human or agent, deepen one deduplicated graph. The contributors do not scatter documents.

The system runs. The three case studies are in the system. The submission includes the snapshot. A judge can open any claim and follow its receipts.

References

[1] Future of Life Foundation. *Lab leaks, black holes, and eggs: epistemic case study competition.* EA Forum, 2026.

[2] P. Miller and S. Wilf. *The Rootclaim COVID-19 origins debate*, structured debate with judge decisions and Bayesian analyses, 2024.

[3] Anthropic. *Model Context Protocol.* `https://modelcontextprotocol.io`, 2024.

[4] J. Priem, H. Piwowar, R. Orr. *OpenAlex: a fully-open index of scholarly works.* arXiv:2205.01833, 2022.

[5] S. Wojcik et al. *Birdwatch: crowd wisdom and bridging algorithms can inform understanding and reduce the spread of misinformation.* arXiv:2210.15723, 2022.

A Agent MCP tool reference

The endpoint is `POST /api/mcp/agent/mcp` with `Authorization: Bearer esk_...` (streamable-HTTP MCP). A person mints keys in the app ("Connect an agent"). Each key is a distinct contributor of kind `agent`. The system records the person who minted the key as `on_behalf_of`. Two servers accept read-only requests without authentication: `POST /api/mcp` serves the whole commons, and `POST /api/mcp/<topic>/mcp` serves curated slices.

Read

```
search(query)            -> { results: [{ id, title, url }] }
fetch(id)                -> node as a document: text, verbatim quotes,
                            challenge threads, chain-of-custody receipt
get_claim(id)            -> text, modality, descriptors, evidence quotes,
                            typed relations, challenges, disputeState, receipt
get_hypothesis(id)       -> statement, linked claims (polarity, diagnosticity),
                            communityCredence, challenges, receipt
```

Investigations

```
list_investigations()    -> all rooms, newest first, with URLs and lineage
create_investigation({ title, seed_from_commons? })
                         -> new room owned by the agent; appears in every
                            member's sidebar live
get_investigation_graph({ investigation_id })
                         -> counts, hypotheses + community credence,
                            openCruxes, node catalog
```

Write (eve's own write path: same dedup, same receipts)

```
record_source({ investigation_id, text, url?, title?, author?,
                publisher?, date? })                    -> { source_id }
record_claim({ investigation_id, claim, source_id, quote,
               descriptors? })   -> { claim_id, is_new, merged_similarity }
                                    # quote MUST appear verbatim in the
                                    # source's stored text (checked in code)
record_relation({ investigation_id, from_claim_id, to_claim_id,
                  type: supports|contradicts|depends_on|refines,
                  rationale? })
record_hypothesis({ investigation_id, statement, answer_bearing? })
link_claim_to_hypothesis({ investigation_id, claim_id, hypothesis_id,
                           polarity: supports|undermines,
                           diagnosticity?: 0..1 })
record_crux({ investigation_id, claim_id, question, implication? })
record_credence({ investigation_id, hypothesis_id, credence: 0..100,
                  rationale? })     # append-only; history = belief timeline
file_challenge({ investigation_id, node_id,
                 challenge_type: counter_evidence|rival_interpretation|
                                 methodological_objection,
                 body, evidence_url? })
```

Collaborate

```
get_transcript({ investigation_id })     -> per-turn Q/A with the message ids
                                            comment anchors use
add_comment({ investigation_id, body, message_id, quote,
              quote_prefix?, quote_suffix? })   # anchored to a quote span
reply_to_comment({ comment_id, body })
send_message({ investigation_id, message, wait_for_reply? })
    # takes a REAL eve turn under the agent's name; room members watch it
    # stream exactly as they would a human member's turn
delegate_investigation({ investigation_id, brief })   -> first narration beats
delegate_step({ investigation_id, delegation_id })    -> advance one phase;
```

```
                                                    done=true at the end
```

Every write fires an activity broadcast. Clients derive agent presence and a live agent cursor from the recency of these broadcasts. An idle agent expires after ∼75 s. Every write creates a `contributions` row. The row records the contributor, the method@version, the payload hash, and the session and turn. The row has the same shape for agent writes, human writes, and eve writes.

B Worked example: one claim's chain of custody

This appendix shows the provenance panel of one claim. The receipts query resolves the panel. The shape comes verbatim from the system. We shortened the content:

```
claim: "..." (canonical id = hash of normalized text)
  created by  eve (agent) . method record_claim@1 . <timestamp>
              during a turn asked by <human contributor>
              in investigation "<title>"
  payload hash  sha256:...          (optional signature over the hash)
  evidence
    mention #1  source <id> "<source title>" (content-addressed)
                quote: "<verbatim passage, code-checked substring of the
                        source's stored full text>"
                retrieval: { operator: delegated_read@1, query: "...",
                             avenue: "...", via: tavily-extract,
                             delegator: <human> }
    mention #2  source <id> ...    # same claim found again elsewhere:
                                   # "many sources, one claim"
  relations   <- supports  claim <id>   "Answers the crux question: ..."
  challenges  1 open (rival_interpretation) -> derived state: contested
  credences   (on the hypothesis it supports) per-assessor timelines
```

A correction never changes this record. A superseding contribution points to the contribution that it corrects. Readers see both contributions.

C Judge runbook

The submission repository contains the full instructions in `README.md`. The repository is a public fork with the secrets removed. The summary follows:

1. Install the prerequisites: Bun, Node ≥ 24, Docker (for local Supabase), and the Supabase CLI.
2. Run `bun install`, then `supabase start`. Restore the included case-study snapshot with one `psql` command. The README gives the exact line. Copy `.env.example` to `.env`. The file needs two keys: a model API key and a search API key. Run `bun run dev`.
3. Open the three case-study investigations. Inspect the provenance panel of any node. Scrub the belief timeline. Open a release page and its `/export` URL.
4. Delegate a brief on a question of your own. Watch the run write into the graph.
5. Mint an agent key in the app. Connect any MCP client to write to the same graph. §5.2 gives the config block.

D Demo video index

One walkthrough video accompanies the submission. The ▶ boxes in the text mark the relevant timestamps. *(The submission form and the README give the link and the timestamps.)*